

Сборка приложения с санитайзером

Сборка сторонних библиотек с санитайзером

Реализована централизованная сборка сторонних библиотек с санитайзером на [Jenkins](#).

Принцип работы

Сборка библиотек с санитайзером проводится в отдельные каналы сборки `asan` и `tsan`. Они не пересекаются с основным каналом `stable`. То есть данные сборки содержат полный набор библиотек и все рецепты должны поддерживать эти сборки.

Для сборки с санитайзером реализован перехватчик (обертка) компилятора ее нужно указывать в рецептах в качестве компилятора (в большинстве случаев подтягивается автоматически из переменных окружения).

Такой подход был выбран, так как:

1. большинство библиотеки не знают о наличие санитайзера.
2. часть библиотек не позволяют легко установить дополнительные флаги компиляции.
3. рецепты не усложняются дополнительными ветвлениями.
4. при добавлении новых рецептов высок шанс, что сборка с санитайзером заработает автоматически.

Обертка компилятора добавляет в командную строку компилятора флаги компиляции и линковки с санитайзером, а также удаляет некоторые несовместимые флаги, для windows дополнительно копирует библиотеки в папку с исполняемыми файлами.

Исходники обертки располагаются [здесь](#).

Варианты сборки

1. `asan+ubsan` (все архитектуры)

[Address sanitizer](#) и [Undefined Behavior Sanitizer](#) (добавляются флаги `-fsanitizer=address` `-fsanitizer=undefined` - `shared-libsan` `-fno-omit-frame-pointer`).

Собирается с динамической библиотекой, необходимо поставлять вместе с приложением.

2. `tsan` (только Linux)

[Thread Sanitizer](#) (добавляются флаги `-fsanitizer=thread` `-fno-omit-frame-pointer`).

Собирается со статической библиотекой санитайзера, которая целиком влинкована в исполняемые файлы (не работает с тестами на Python).

Опции рецепта

Есть несколько библиотек которые знают о наличии санитайзера и требуют передачи дополнительных флагов в тулчейн сборки, а также есть библиотеки для которых сборка с санитайзером приводит к ошибкам самого компилятора или нам не нужны санитизированные версии этих библиотек.

1. Возможность выключить, урезать сборку с санитайзером для конкретного рецепта - атрибут `supported_extra_tools` для `conanfile.py` управляет работой обертки компилятора, заменяет исходный параметр. Например:

```
1 # полностью отключает добавление флагов санитайзера для обоих
  вариантов сборки.
2 supported_extra_tools = {"asan+ubsan": "", "tsan": ""}
1 # отключает только добавление флага -fsanitize=undefined.
2 supported_extra_tools = {"asan+ubsan": "asan"}
```

2. Возможность добавить кастомные настройки - переменная окружения `SBIS_EXTRA_TOOL` - имеет значение соответствующие применяемым флагам. Например:

```
1 SBIS_EXTRA_TOOL="" - для обычной сборки.
2 SBIS_EXTRA_TOOL="asan+ubsan" - для варианта с Address sanitizer.
```

```

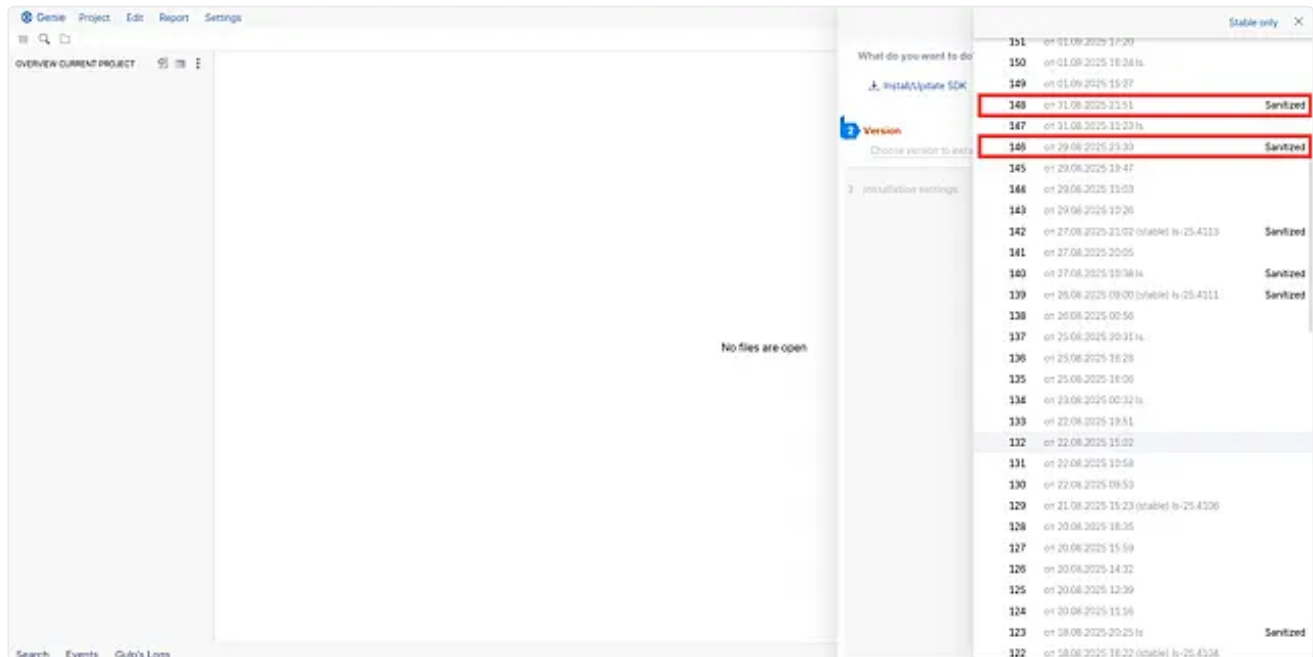
1 if os.environ.get("SBIS_EXTRA_TOOL", "") == "tsan":
2     flags.append("context-impl=ucontext")
3     flags.append("define=BOOST_USE_TSAN")

```

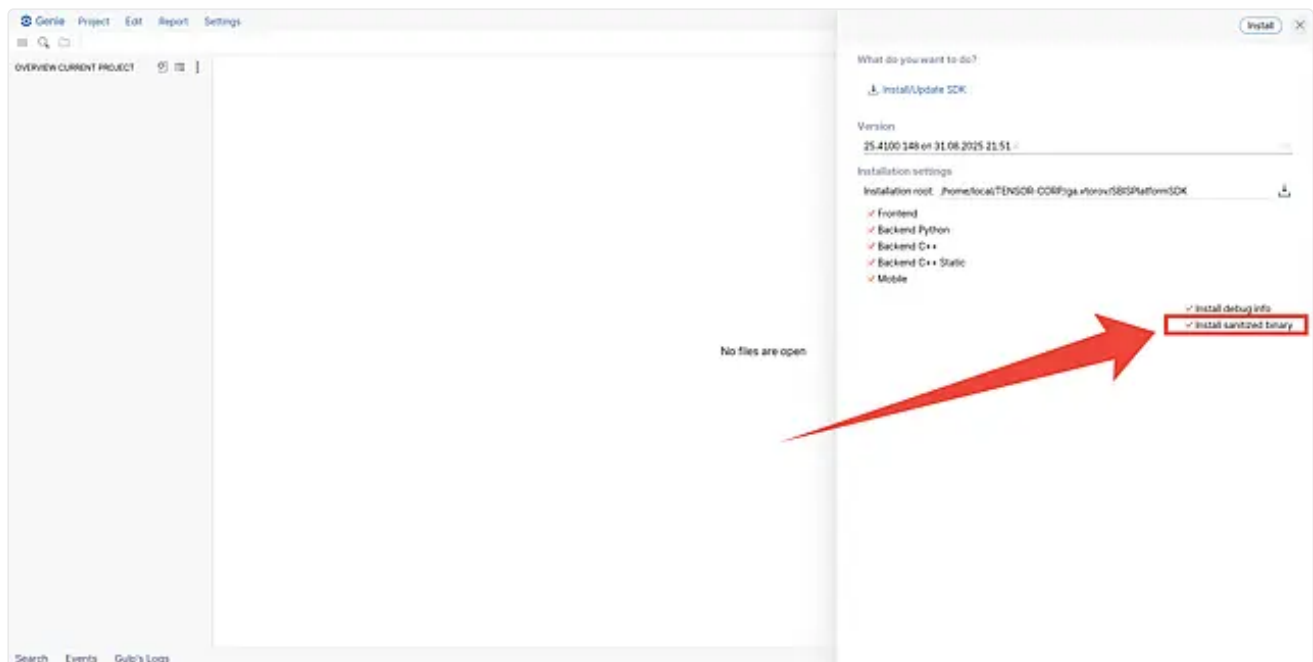
Поставка SDK платформы с санитайзером

Реализована централизованная сборка SDK с санитайзерами на [Jenkins](#), на данный момент она используется для сборок мобильных приложений и online с санитайзером и запуском на них автотестов.

Также санитизированные версии библиотек можно получить через Genie при установке SDK часть билдов помечена флагом Sanitized.



для них доступна опция установки **Install sanitized binary**.



Дополнительные бинари поставляется для всех доступных конфигураций сборки, для Linux они будут располагаться в "bin/linux/centos7/x86_64/clang15/asan".

Сборка приложения (в том числе SDK платформы) с санитайзером

Полная сборка

Для сборки приложения с санитайзером с использованием санитизированных сторонних библиотек и СДК необходимо передать параметр в командную строку запуска stake:

```
1 -DSBIS_CONFIGURATION=asan
```

или *tsan*.



При этом в логах будет следующая запись:

```
-- Configuration: asan
```

Частичная сборка

Для сборки только самого приложения без зависимостей необходимо передать параметр в командную строку stake:

```
-DSBIS_SABITIZE=address
```

или *undefined, thread, asan+ubsan*.



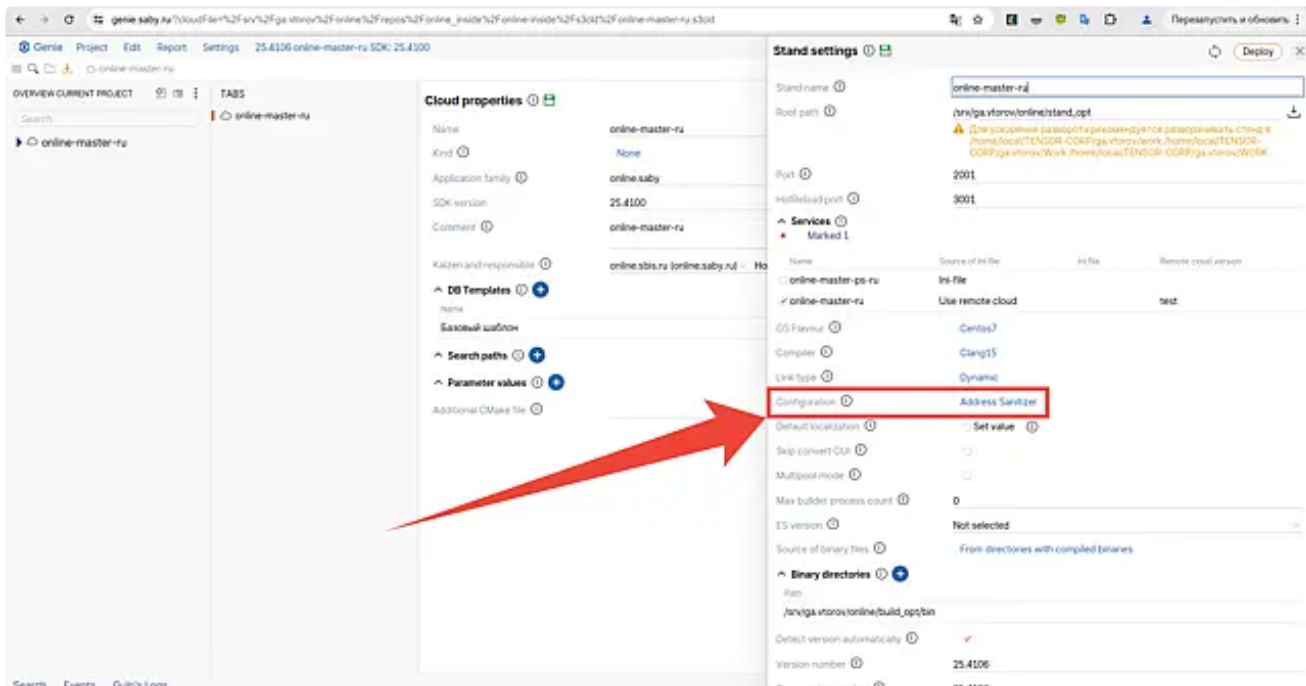
При этом в логах будет следующая запись

```
-- Using "address" sanitizer for C and C++ compilers
```

Разворот приложения с санитайзером (кроме мобильных приложений)

Полная сборка

Доступен автоматический разворот санитизированного приложения через Genie в настройках разворота появился новый параметр `configuration` в нем необходимо выбрать `Address Sanitizer(Thread Saniaizer)`.



Для jinnee_utility прописать в s3distr файле параметр configuration="asan" (tsan).

Частичная сборка

Параметр configuration изменять не нужно оставить release, но для asan нужно будет скопировать библиотеку санитайзера в каталог развернутого приложения.

Linux:

`/opt/clang-15.0.6/lib64/clang/15.0.6/lib/libclang_rt.asan-x86_64.so`

Macos:

`/Applications/Xcode.app/Contents/Developer/Toolchains/XcodeDefault.xctoolchain/usr/lib/clang/16.0.0/lib/darwin/libclang_rt.asan_osx_dynamic.dylib`

Windows:

`C:\Program Files (x86)\TensorCppDev-2022\llvm\lib\clang\20\lib\windows\clang_rt.asan_dynamic-i386.dll`

`C:\Program Files (x86)\TensorCppDev-2022\llvm\lib\clang\20\lib\windows\clang_rt.asan_dynamic-x86_64.dll`

Запуск приложения с санитайзером(кроме мобильных приложений)

Для корректной работы санитайзера необходимо задать ряд переменных окружения

Общие

Добавить в PATH путь до llvm_symbolizer:

- Linux:

`/opt/llvm-15.0.6/bin`

- macOS:

необходимо дополнительно установить, например, через brew:

`brew install llvm`

тогда путь будет

/usr/local/opt/llvm/bin

- Windows:

C:\Program Files (x86)\TensorCppDev-2022\Llvm\bin

Address sanitizer

- Минимальный набор:

```
1 LD_PRELOAD=<путь до библиотеки санитайзера>
2 ASAN_OPTIONS=log_path=<путь куда писать
  отчеты>:external_symbolizer_path=<путь до llvm_symolizer>
```

- Известные проблемы (добавляются к минимальному набору):

```
1 LD_PRELOAD=$LD_PRELOAD:/usr/lib64/libresolv.so.2
2 ASAN_OPTIONS=$ASAN_OPTIONS:alloc_dealloc_mismatch=0:detect_leaks=0:de
  tect_odr_violation=0:report_globals=0
```

- Дополнительные флаги (больше проверок):

```
ASAN_OPTIONS=$ASAN_OPTIONS:redzone=1024:quarantine_size_mb=256:strict_str
  ing_checks=1:check_initialization_order=1:detect_stack_use_after_return=1:
  strict_init_order=1
```

Общие флаги для всех санитайзеров размещены [здесь](#).

Thread sanitizer

- Минимальный набор:

```
TSAN_OPTIONS=log_path=<путь куда писать отчеты>:external_symbolizer_path=
  <путь до llvm_symolizer>
```

- Известные проблемы (добавляются к минимальному набору):

```
TSAN_OPTIONS=$TSAN_OPTIONS:die_after_fork=0
```



Для Windows нет переменной LD_PRELOAD

Запуск стенда с санитайзером через Genie

Для запуска стенда с санитайзером необходимо добавить описанные выше переменные окружения для этого в jinnее на этапе разворота добавлено окно куда их можно вписать.

Stand settings

Deploy

OS Flavour	Centos7						
Compiler	Clang15						
Link type	Dynamic						
Configuration	Address Sanitizer						
Default localization	☐ Set value						
Skip convert GUI	☐						
Multipool mode	☐						
Max builder process count	0						
ES version	Not selected						
Source of binary files	From zip/ Unpacked distribution						
Distribution	/srv/ga.vtorov/online/distr						
Detect version automatically	☒						
Version number	25.6100						
Core version number	25.6000						
Compile and pack BL resources	☐						
After deploy	Nothing						
Full Gulp logs	☐						
Compile UI files on change	☐						
Symlinks only on files	☐						
Environment Variables <table><thead><tr><th>Key</th><th>Value</th></tr></thead><tbody><tr><td>ASAN_OPTIONS</td><td>log_path=/home/asan_report/new:llvm_symbolizer=/opt/llvm-15.0.6/bin</td></tr><tr><td>LD_PRELOAD</td><td>libclang_rt.asan-x86_64.so</td></tr></tbody></table>		Key	Value	ASAN_OPTIONS	log_path=/home/asan_report/new:llvm_symbolizer=/opt/llvm-15.0.6/bin	LD_PRELOAD	libclang_rt.asan-x86_64.so
Key	Value						
ASAN_OPTIONS	log_path=/home/asan_report/new:llvm_symbolizer=/opt/llvm-15.0.6/bin						
LD_PRELOAD	libclang_rt.asan-x86_64.so						
Choose Compiled UI Resources	☒						

Environment variable

в этом случае в папке стенда будет создан файл .env, в котором будут сохранены эти переменные и использованы jinnee-utility при запуске стенда.